

Lab5实验报告

思考题

Thinking 5.1

/proc文件系统是一种特殊的，由软件创建的（伪）文件系统，内核使用它向外界导出信息，/proc系统只存在内存当中，而不占用外存空间。/proc下面的每个文件都绑定于一个内核函数，用户读取文件时，该函数动态地生成文件的内容。

与其它常见的文件系统不同的是，/proc是一种伪文件系统（也即虚拟文件系统），存储的是当前内核运行状态的一系列特殊文件，用户可以通过这些文件查看有关系统硬件及当前正在运行进程的信息，甚至可以通过更改其中某些文件来改变内核的运行状态。

Windows，分盘，每个驱动器有自己的根目录，形成的是多个树并列的结构。

Linux，只有一个根目录 /，所有东西都是从这开始

Thinking 5.2

kseg0存储了内核，且通过cache访问内核。如果对设备的写入缓存到cache中就会导致访问内核时访问了设备的写入内容

Thinking 5.3

mos文件控制块

```
struct File {
    u_char f_name[MAXNAMELEN]; // 文件名字
    u_int f_size; // 文件大小
    u_int f_type; // 文件类型
    u_int f_direct[NDIRECT]; // 文件直接指针
    u_int f_indirect; // 文件间接指针
    struct File *f_dir; // 指向文件所属的文件目录（当此文件控制块可用时）
    u_char f_pad[BY2FILE - MAXNAMELEN - 4 - 4 - NDIRECT * 4 - 4 - 4]; // 为了让整个文件结构体占用一个磁盘块，填充结构体中剩下的字节
};
```

Unix/Linux操作系统inode:

```
struct m_inode {
    unsigned short i_mode; /* 文件类型和属性，ls查看的结果，比如drwx----- */
    unsigned short i_uid; /* 文件宿主id */
    unsigned long i_size;
    unsigned long i_mtime; /* 文件内容上一次变动的时间 */
    unsigned char i_gid; /* groupid: 宿主所在的组id */
    unsigned char i_nlinks; /* 链接数: 有多少个其他的文件夹链接到这里 */
    unsigned short i_zone[9]; /* 文件映射的逻辑块号 */
    /* these are in memory also */
    struct task_struct * i_wait; /* 等待该inode节点的进程队列 */
    unsigned long i_atime; /* 文件上一次打开的时间 */
    unsigned long i_ctime; /* 文件的inode上一次变动的时间 */
};
```

```

unsigned short i_dev; /*设备号*/
unsigned short i_num;
/* 多少个进程在使用这个inode*/
unsigned short i_count;
unsigned char i_lock; /*互斥锁*/
unsigned char i_dirt;
unsigned char i_pipe;
unsigned char i_mount;
unsigned char i_seek;
/*
数据是否是最新的，或者说有效的，
update代表数据的有效性，dirt代表文件是否需要回写，
比如写入文件的时候，a进程写入的时候，dirt是1，因为需要回写到硬盘，
但是数据是最新的，update是1，这时候b进程读取这个文件的时候，可以从
缓存里直接读取。
*/
unsigned char i_update;
};

```

MOS系统对文件的操作依靠进程间通信来完成，而Linux直接通过系统调用来完成

Thinking 5.4

一个磁盘块最多存储16个文件控制块，单个文件最多有1024个指针，指向1024个磁盘块，因此一个目录下最多16384个文件

Thinking 5.5

在fs/fs.h的宏定义中可以看出我们实验使用的内核支持的最大磁盘大小是1GB

Thinking 5.6

不能正常工作，因为系统从DISKMAP即0x10000000开始映射缓存的磁盘块，缓存的磁盘块保存在serv.c这个用户的内存空间中，而serv.c进程会从FILEVA即0x60000000开始为Open结构分配空间，在DISKMAX大于0x50000000后映射的磁盘块就会覆盖掉Open结构，而在DISKMAX大于0xB0000000便超过了用户空间，会导致文件控制进程试图访问内核数据，引发异常并panic，系统无法正常运行

Thinking 5.7

```

// 文件控制块定义
struct File {
    u_char f_name[MAXNAMELEN]; // 文件名字
    u_int f_size; // 文件大小
    u_int f_type; // 文件类型
    u_int f_direct[NDIRECT]; // 文件直接指针
    u_int f_indirect; // 文件间接指针
    struct File *f_dir; // 指向文件所属的文件目录（当此文件控制块可用时）
    u_char f_pad[BY2FILE - MAXNAMELEN - 4 - 4 - NDIRECT * 4 - 4 - 4]; // 为了让整
    数个文件结构体占用一个磁盘块，填充结构体中剩下的字节
};

// 各种操作对应的标志数
#define FSREQ_OPEN 1
#define FSREQ_MAP 2
#define FSREQ_SET_SIZE 3
#define FSREQ_CLOSE 4
#define FSREQ_DIRTY 5

```

```

#define FSREQ_REMOVE 6
#define FSREQ_SYNC 7
struct Fsreq_open { // 打开文件
char req_path[MAXPATHLEN];
u_int req_omode;
};
struct Fsreq_map { // 定位文件
int req_fileid;
u_int req_offset;
};
struct Fsreq_set_size { // 改变文件大小
int req_fileid;
u_int req_size;
};
struct Fsreq_close { // 关闭文件
int req_fileid;
};
struct Fsreq_dirty { // 将文件标记为已修改
int req_fileid;
u_int req_offset;
};
struct Fsreq_remove { // 删除文件
u_char req_path[MAXPATHLEN];
};

```

Thinking 5.8

user/file.c中的struct Fd*指针都是通过open函数执行得到的，而open调用了fsipc_open函数，并将一个struct Fd型的指针的值发送给serv；serv会用ipc将Fd指针的所在页映射上一个struct Filefd；而Filefd中第一个结构体成员便是Fd结构体，因此处于Filefd结构体中的Fd结构体的指针所指向的地址与其所处的Filefd结构体指针所指向的地址是相同的，因此可以直接转化

Thinking 5.9

由于文件描述符和定位指针均存储在用户空间，因此fork前后的父子进程会共享文件描述符和定位指针程序可由lab5-2的exam课上第二题轻易验证

Thinking 5.10

struct Fd定义在user/fd.h，是一个文件描述符结构，是库函数保存用户进程已打开文件使用的。

- fd_dev_id：打开文件的 id，也就是该文件描述符对应的抽象文件的实际类型，指示了该文件所处的设备
- fd_offset：当前读/写的偏移值，也就是下一次操作从文件的哪个地方开始
- fd_omode：当前文件打开的模式（访问权限），只读/只写/读写等，可在判定操作是否合法时用。

struct Filefd定义在user/fd.h，是文件描述符与文件id与文件控制块的结构

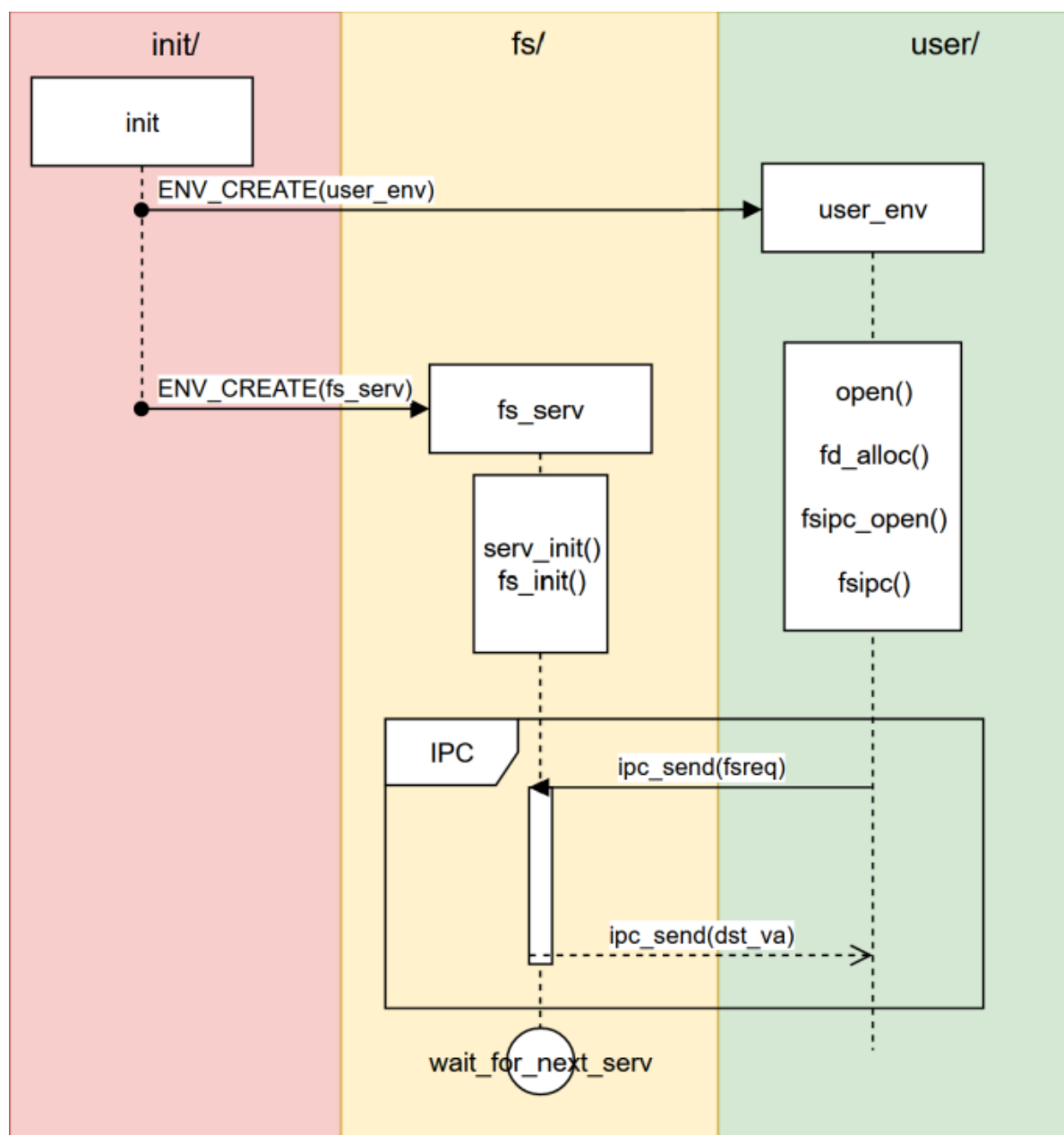
- f_fd：一个文件描述符。
- f_fileid：对应于一个全局的文件编号，用来向文件系统请求服务。
- f_file：对应文件的文件控制块。

struct Open定义在fs/serv.c，是文件系统服务用来保存整个系统的已打开文件的结构。

- o_file：真实的，指向对应文件在硬盘块缓存上文件控制块的地址，用来对文件进行属性进行更改
- o_fileid：全局唯一的文件编号 id，和 struct Filefd里的 f_fileid对应。

- o_mode: 文件打开的模式, 和 struct Fd 的 fd_omode对应
- o_ff: 文件读/写偏移量, 即文件读/写当前位置

Thinking 5.11



黑色实线箭头是同步消息

黑色虚线箭头是返回消息

返回消息和同步消息结合使用, 使用同步信息时, 消息的发送者把进程控制传递给消息的接收者, 然后暂停活动, 等待消息接收者的返回消息

而使用IPC进行进程间通讯使用的是fsipc函数

Thinking 5.12

因为该进程在调用sys_ipc_rcv是会被设置为ENV_NOT_RUNNABLE并调用sys_yield主动让步, 只有接收到了信息才会重新变为ENV_RUNNABLE, 因此不会导致整个内核进入panic状态

练习题

Exe 5.1

步骤：1. 判断地址是否合法

2. 用bcopy虚拟地址的内容与kseg1的读写，用dev+0xA0000000来进行物理地址到kseg1内核地址的转换

Exe 5.2

参考前面内核部分的驱动函数read_sector，用户态的步骤完全一致，只是需要调用sys_write_dev和sys_read_dev而非直接操作地址

```
1  # read sector at specified offset from the beginning of the disk image.
2  LEAF(read_sector)
3      sw a0, 0xB3000010 # select the IDE id.
4      sw a1, 0xB3000000 # offset.
5      li t0, 0
6      sb t0, 0xB3000020 # start read.
7      lw v0, 0xB3000030
8      nop
9      jr ra
10     nop
11 END(read_sector)
```

Exe 5.3

blockno除以32求出对应数组的第几位，再对32取模得到是32位上的第几位并且置1即可

体会与感想

本次实验难点在于理解各个结构体及其结构体成员以及宏定义的含义，需要阅读的代码量很大，且许多定义在指导书上讲解的不是很详细。因此需要耐心在文件中寻找阅读相关代码，本次实验学习到了文件系统的基本概念以及普通磁盘的基本结构和读写方式，并通过系统驱动掌握并实现了文件系统服务的基本操作。